

01 / TEAM DISCUSSION GUIDE

CNN implementation & team project plan

COMP 6701 - Project 7: Implementing a CNN and investigating convolution
Three-person team | Prepared for discussion | 10 October 2026

Proposed research question

With approximately matched parameter counts and training budgets, how do a CNN and an MLP compare on MNIST and Fashion-MNIST, and how does pooling affect accuracy and sensitivity to image translations and noise?

Required by the supplied course guidelines

Requirement	What the team must produce
Core implementation	A LeNet-style CNN and a reasonably matched fully connected baseline, written directly using JAX primitives.
Permitted implementation stack	Python and its standard library, NumPy, SciPy, Matplotlib and/or JAX. JAX automatic differentiation is allowed for this project.
Data	Experiments on both MNIST and Fashion-MNIST, with documented data preparation and evaluation splits.
Correctness and comparisons	A working baseline, correctness checks, accuracy and parameter comparisons, learning curves, and controlled comparisons of architectural components.
Robustness and visual analysis	Tests with small translations and noise; visualizations of learned filters and intermediate feature maps.
Reproducibility and teamwork	Git version control, reproducible instructions and configurations, and explicit individual contributions.
Final submission	Code, reproducibility instructions, a four-page report written in LaTeX and compiled to PDF, and a short recorded presentation.

Scope to agree in the meeting

Complete the core implementation and required experiments first. Residual connections and deeper models are optional extensions. CPU is sufficient for the core according to the guidelines; benchmark the actual setup before scheduling repeated runs.

Status: This is a planning document, not the final four-page report. Dates, three-seed runs, architecture details and the suggested parameter-matching tolerance are team proposals. The guidelines say mid-December; confirm the exact deadline and recording duration.

02 / TEAM DISCUSSION GUIDE

Fair ownership & collaboration

Replace Member A, B and C with names during the meeting. Each person owns implementation, checks, experiments and writing within their area.

Owner	Responsibilities and concrete handoff
Member A CNN and architecture	Build: Convolution and pooling layers, the CNN, and pooling variants. Check: Layer calculations, shapes and selected gradients. Investigate: Pooling comparisons; learned filters and feature maps. Deliver: Tested CNN modules, architecture configurations, figures and an explanation of differences from the original LeNet. Write/present: Architecture and feature analysis.
Member B Training and baseline	Build: MLP baseline, loss, optimizer, shared training loop and checkpoint save/load. Check: Tiny-set learning, loss reduction and checkpoint round-trips. Investigate: Clean-image CNN/MLP comparisons, parameter counts and learning curves. Deliver: One trainer that accepts either model, training logs and comparison figures. Write/present: Training method and baseline comparison.
Member C Data and evaluation	Build: Data loading, fixed splits, batches, metrics, translation/noise transforms and result aggregation. Check: Data/label alignment, split separation, metrics and transforms. Investigate: Accuracy degradation under translations and noise. Deliver: Shared data/evaluation modules, robustness results and a draft execution guide. Write/present: Data, evaluation design and robustness analysis.

Working agreements

One owner, one reviewer. A reviews B, B reviews C, and C reviews A. Keep pull requests small enough for a teammate to understand.

Agree on interfaces first. Discuss changes to data shapes, model outputs, configurations and checkpoint format before implementing them.

Balance effort weekly. Count debugging, review, experiments and writing as work. Compare estimated and actual effort; redistribute remaining tasks if one person is overloaded.

Escalate blockers early. Tell the team what is blocked, what has been tried and what help is needed. Use a short weekly check-in to resolve dependencies.

Share integration and delivery. Each owner writes their section and figure captions. Assign final editing, a clean-environment rerun and recording coordination explicitly; rotate coordination where useful.

Definition of done: The change runs, passes relevant checks, has a usage note and saved evidence where applicable, is reviewed by another member, and is merged into the shared codebase.

03 / TEAM DISCUSSION GUIDE

Implementation roadmap

Proceed through these checkpoints in order. Data work, CNN layers and baseline/trainer development can begin in parallel after the shared interfaces are agreed.

1. Agree on the scope and working setup

Confirm the research question, owners, repository, available compute and weekly availability. Read the LeNet reference and record which architectural ideas will be retained or changed.

Done when: A short team agreement exists and everyone knows the first deliverable and review date.

2. Define the shared code contract

Agree on image layout, model function signatures, configuration fields, metrics and checkpoint format. Use the interface checklist on page 4.

Done when: A and B can plug their models into the same trainer; C can test evaluation using known mock outputs.

3. Build a small end-to-end baseline run

C supplies a small MNIST subset and batches. B trains an MLP and saves logs and weights. A implements CNN layers and their checks in parallel.

Done when: An MLP run completes, produces sensible metrics, and can reload its saved parameters.

4. Integrate the CNN and verify correctness

Connect the CNN to the same training/evaluation path. Check layer outputs, selected gradients and the ability to fit a tiny fixed dataset.

Done when: Both models train and pass the agreed checks before longer runs start.

5. Freeze the main experimental protocol

Select a small CNN, match MLP capacity reasonably, define pooling variants and measure run time. Fix data splits, budgets, tuning rules and evaluation transformations.

Done when: Configurations and a feasible run matrix are committed; test data has not been used for tuning.

6. Run and record the comparisons

B leads clean-image comparisons, A leads pooling and feature analysis, and C leads robustness evaluation. All owners save configurations, seeds, metrics and checkpoints.

Done when: Every planned comparison has traceable outputs or a documented failure.

7. Analyze and write together

Each member drafts the method, results and limitations for their work. Discuss variability, confounding factors and negative outcomes before writing conclusions.

Done when: The four-page report has coherent figures, cautious conclusions and an accurate contribution statement.

8. Reproduce and prepare the final submission

Follow the README from a clean environment, check the submission checklist, and record a short presentation in which all members explain their own work.

Done when: The documented commands work and the complete package is ready before the confirmed deadline.

04 / TEAM DISCUSSION GUIDE

Shared interfaces & correctness

Set these conventions before separate development begins. The values below are proposed starting conventions, not requirements imposed by the instructor.

Item	Proposed agreement
Images and labels	Images: float32, shape (batch, 28, 28, 1), scaled to [0, 1]. Labels: integers 0-9. The MLP flattens images internally.
Model API	init_params(key, config) creates parameters. forward(params, images, config) returns logits with shape (batch, 10). Use a separate feature-inspection function so the trainer interface stays stable.
Training API	Use the same loss, optimizer implementation and training path for all variants. A simple optimizer implemented directly in JAX keeps the core manageable.
Configuration	Record model, dataset, split seed, initialization seed, shuffle seed, evaluation seed, learning rate, batch size and epoch/update budget.
Results and checkpoints	Save per-epoch train/validation loss and accuracy, model configuration, parameter count and code commit. Agree on a NumPy-compatible checkpoint format and a JSON/CSV log format.
Implementation building blocks	Convolution: jax.lax.conv_general_dilated. Pooling: suitable jax.lax.reduce_window reductions. Automatic gradients: jax.grad or jax.value_and_grad. Specify image/kernel dimension conventions explicitly.

Correctness checklist before long runs

Check	Evidence to keep
Convolution and pooling	Use small inputs with hand-computed or simple NumPy reference outputs. Match the intended kernel convention, stride and padding.
Shapes and labels	Verify every layer shape, data/label alignment and the ten-class output. Check train/validation/test separation.
Selected gradients	Compare automatic gradients against central finite differences on a tiny example. Avoid ReLU kinks and max-pooling ties; choose numerical precision and tolerance deliberately.
Tiny-set learning	Try to fit 32 or 64 fixed samples with both models. A failure to reduce loss is a debugging signal, not an experimental conclusion.
Checkpoints and metrics	Predictions should agree before and after save/load. Verify accuracy on known logits and labels.
Image transformations	Inspect transformed samples. Check shift direction, zero-filled borders, noise strength and clipping to the chosen pixel range.

Keep concise evidence of these checks in the repository. A small set of meaningful checks is more useful than many tests that simply repeat the implementation.

05 / TEAM DISCUSSION GUIDE

Controlled experiment plan

Start with a compact comparison that answers the project question. Add experiments only after the required runs work and the team has reviewed the compute budget.

Model	Purpose
MLP baseline	Flattened images, fully connected hidden layers and ten logits. Adjust width to roughly match the CNN parameter count.
CNN with max pooling	Small LeNet-style network: two convolution/ReLU/pooling blocks, followed by a compact fully connected classifier.
CNN with subsampling	Replace each pooling operation with fixed stride-based pixel selection, preserving output shape and downstream parameter count.

Suggested core budget: 3 model variants x 2 datasets x 3 training seeds = 18 training runs. Apply translation and noise tests to these saved models; no new training is required for those evaluations. Pilot one short run before committing to this budget.

Comparison rules

Match capacity and training opportunity. Aim for parameter counts within roughly 10% for the MLP/CNN comparison. Use the same data, batch size and training budget. If tuning learning rates, give models the same search budget.

Control pooling variants closely. Keep CNN layer widths, initial parameters, data order and training settings matched between pooling variants. Compare max aggregation with stride-based selection at the same reduction factor.

Reserve the test split. Choose settings and checkpoints using validation data, then evaluate the fixed protocol on held-out test data. Publish split indices or their deterministic generation rule.

Report all planned runs. Plot train/validation loss and accuracy; report final test accuracy, parameter count, individual seed results and mean with standard deviation.

Evaluation	Fixed procedure and output
Clean images	Evaluate all variants on both datasets. Report accuracy and learning curves; explain gaps between training and validation.
Small translations	Test 1- and 2-pixel shifts in specified directions. Fill empty borders with zeros; never wrap pixels to the opposite side. Report accuracy and percentage-point loss relative to clean images.
Noise	For pixels in $[0, 1]$, a proposed Gaussian-noise grid is standard deviation 0.05, 0.10 and 0.20. Clip to $[0, 1]$. Reuse the same noisy images across models and record the evaluation seed.
Filters and feature maps	Use a fixed selection rule and the same example images across relevant comparisons. Include readable scales and captions explaining what the visualization can support.

Interpretation limits: Removing pooling outright changes feature-map sizes and often model capacity. The proposed substitution tests max aggregation versus sample selection. CNN/MLP differences involve several architectural choices; parameter matching alone does not isolate weight sharing. Pooling does not guarantee translation invariance.

06 / TEAM DISCUSSION GUIDE

Milestones & final handoff

Suggested schedule from 10 October to mid-December 2026. Adjust after confirming the exact submission date and measuring training time.

Target window	Milestone / exit condition
10-16 October	Team agreement, Git repository, shared interfaces and accessible datasets.
17-30 October	Small end-to-end MLP and CNN runs; shared training/evaluation path works.
31 October-6 November	Correctness checks complete; architectures, tuning rules and run matrix fixed.
7-20 November	Main training runs and pooling comparisons completed and recorded.
21-27 November	Robustness tests, filter/feature-map figures and joint interpretation completed.
28 November-4 December	Complete report draft, contribution statement and reproducibility check.
5-12 December	Fix remaining issues, record the presentation and prepare the final package.

Suggested four-page final report structure

Page	Main content
1	Research question, datasets, model architectures and differences from LeNet.
2	Correctness evidence, training settings and controlled experimental design.
3	Main results with carefully selected comparison plots and robustness results.
4	Interpretation, limitations, conclusions and individual contributions.

Check the course formatting rules, including how references count toward the four-page limit. Each owner supplies concise text and captions; one editor coordinates consistency, with all members reviewing the final version.

Reproducibility and submission checklist

Code and setup: Pin dependency versions; document the hardware/backend and the exact commands for a quick check, training, evaluation and figure generation.

Data and randomness: Document sources, preprocessing, split generation and all random seeds. If using a subset, fix and justify the sampling rule.

Traceable results: Retain configs, logs and required checkpoints. Tie reported results to the relevant code version; do not replace failed runs silently.

Clean rerun: A teammate follows the README in a fresh environment and records any reproducibility gaps before submission.

Final package: Include code, instructions, the LaTeX source and compiled four-page PDF, and the short recorded presentation, subject to the course submission format.

Success criterion: Correct implementation, controlled experiments, reproducibility and clear interpretation. High benchmark scores or positive results are not prerequisites for a strong project under the supplied guidelines.

07 / TEAM DISCUSSION GUIDE

First meeting worksheet

Use this page to turn the proposed plan into agreed owners, interfaces and the next concrete deliverables. Suggested meeting length: 35-45 minutes.

Topic	Decision to reach
Scope and question (5 min)	Confirm the core question, both datasets and the three proposed model variants.
Ownership and effort (10 min)	Assign A/B/C, reviewers, weekly availability and the first coordination role.
Interfaces and compute (10 min)	Agree on API/data contracts, the pilot machine and how experiment outputs will be shared.
Protocol and next steps (10-20 min)	Approve the initial run budget, set the first integration date and assign actions below.

Record the team decisions

Decision	Agreed value
Member A / CNN owner	
Member B / training owner	
Member C / evaluation owner	
Repository and shared result location	
Pilot compute / person running it	
First integration date / weekly check-in	
Confirmed deadline / video duration	

Actions before the next meeting

Owner	Concrete deliverable	Due date
A	CNN interface draft and small convolution/pooling checks.	
B	MLP and trainer skeleton with a tiny synthetic-data run.	
C	Dataset loader, fixed split proposal and sample batches.	

Sources and implementation references

Course scope: supplied COMP 6701 Fall 2026, "Group project ideas and guidelines," Project 7 and general requirements. The course source defines the required deliverables; this document proposes the execution plan.

Architecture reference: LeCun et al., Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 1998.

Official JAX documentation: [convolution primitive](#) | [window reductions for pooling](#) | [automatic differentiation](#).